

Operating Systems

Lab experience - File Control

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

Goals of this lab experience:

- Getting used to non-blocking file operation
- Getting used to multiplexing through `poll()`
- Getting used to lock files to synchronize access to shared resources

- Review the source files `blocking-server.c` and `blocking-client.c` that you can find in
Moodle → Esercitazioni e dimostrazioni → Laboratorio file control
- The server creates and opens `NCLIENTS` fifos, and waits for clients to connect by opening the fifos for writing. Clients send numbers to the server through the fifos. The server sums all numbers received by all clients. When all client close their fifos, the server prints the final sum and terminates.
- Read it, understand how it works, compile and run it, check that it works as expected

The Poll() Awakens

Exercise 1: non-blocking and poll()

- Modify `blocking-server.c` to remove blocking reads on the fifos, and to use `poll()` to wait for data to be available on any of the fifos.
- You can draw inspiration from the code of `07-poll-server.c` that you can find in Moodle → Esercitazioni e dimostrazioni → File control
- Hints:
 - No need to change `blocking-client.c`, keep it as it is
 - Since `read()` is non-blocking, it will no longer return 0 when the client closes the fifo. You can use the `POLLHUP` event to detect when a client has closed the fifo.
 - It might be that a client sends multiple numbers before we `read()` from the fifo. Make sure you read all available data from the fifo before moving on to the next one.

Exercise 1: hints

- No need to change blocking-client.c, keep it as it is
- Since read() is non-blocking, it will no longer return 0 when the client closes the fifo. You can use the POLLHUP event to detect when a client has closed the fifo.
- It might be that a client sends multiple numbers before we read() from the fifo. Make sure you read all available data from the fifo before moving on to the next one.

Blocking vs non-blocking reads and writes on fifos

- Differences between blocking and non-blocking calls to read() on fifos:

Scenario	Blocking	Non-blocking
Data available	Returns bytes read	Returns bytes read
Nothing to read	Blocks until data available	Returns -1, errno=EAGAIN
No writer process	Returns 0 (EOF)	POLLHUP event

- Differences between blocking and non-blocking calls to write() on fifos:

Scenario	Blocking	Non-blocking
Buffer available	Returns bytes written	Returns bytes written
Buffer full	Blocks until space available	Returns -1, errno=EAGAIN
No reader process	Blocks (or SIGPIPE)	Returns -1, errno=EPIPE

Revenge of the Fork()

Exercise 2: multiprocess server

- Modify `blocking-server.c` fork a child for each of the fifos. Each child should read from its fifo (blocking reads are perfectly fine, since there is a single fifo for each process).
- The server also creates a shared memory area that keeps the accumulator.
- Each child process should read a number from its fifo and add it to the accumulator in the shared memory area. Then it resumes reading and stops when the client closes the fifo.
- The server should wait for all child processes to terminate, then print the final value of the accumulator and terminate.

Exercise 2: hints

- Let the server create and open all fifos before forking the children. This way, the children can simply inherit the file descriptors of the fifos from the parent, and they don't need to know which fifo they should open.
- Loop over the fifos and fork a child for each of them. Each child should read from its fifo and add the numbers to the accumulator in the shared memory area. When the client closes the fifo, the child should terminate.
- After having forked all children, the parent can close all fifos (since it doesn't need them anymore) and wait for all children to terminate.
- The shared memory area can be anonymous, created with `mmap()` and shared between the parent and the children by using the `MAP_SHARED` flag. Make sure to initialize the accumulator to 0 before forking the children.

Exercise 2: race condition!

- Even though everything is correctly implemented, you might notice that the final value of the accumulator is not correct (lower than expected).
- This is because of a race condition on the shared memory area. Multiple child processes might read the value of the accumulator at the same time, add their number to it, and write it back, overwriting each other's updates.

The Phantom Race Condition

Exercise 3: file locks

- Modify the server from Exercise 2 to use file locks to synchronize access to the shared memory area.
- Each child process should acquire an exclusive (write) lock on a lock file before reading and updating the accumulator, and release it afterwards.
- If the file is already locked, the child should wait until the lock is available.
- This is **horribly slow and inefficient**, but it prevents race conditions and lets the server produce correct results.

Exercise 3: hints

- The server should create a single lock file before forking the children, and the children should all use that same file to synchronize access to the shared memory area.
- Childs can use `fcntl()`, `F_SETLK` and `flock` structures to implement the locking mechanism.
- **Note:** this will be painfully slow. It is better to reduce the number of clients and the number of iterations in the client to test it.
- While everything is running, execute the command `top` in the terminal to see who is using resources. Look at `us` (cpu cycles in userspace) and `sy` (cpu cycles in kernel space). Almost all will be in `sy`, because the processes will spend most of their time waiting for the lock to be available, which is a kernel operation.

Exercise 3: different locking

- Start from your solution to Exercise 3, and change the places in which a client acquires/releases the lock.
- Make the child acquire the lock as soon as it starts and release the lock right before it terminates.
- What happens? Is it faster or slower? Is it truly concurrent or sequential? Why?